Flutter App Integration & Teammate Handoff Manual

Complete Plug-and-Play Guide for Connecting Flutter to our LoRa Search & Rescue Nodes

Flutter • Dart | flutter_blue_plus | PIN: 123456

> **For the Flutter Developer:** This document contains everything needed to integrate our 3 live ESP32-C6 LoRa hardware nodes into your Flutter codebase. All files and snippets are pre-tested and ready to drop in.

1. LIVE HARDWARE NODE PROFILES

Hardware Node	BLE Advertising Name	Pairing PIN	Sensors & Peripherals Installed
Node #1	`Meshtastic_9448`	`123456`	SX1262 LoRa (868MHz), NEO-6M GPS, MPU6050, SOS Button, Piezo Siren
Node #2	`Meshtastic_aee8`	`123456`	SX1262 LoRa (868MHz), NEO-6M GPS, MPU6050, SOS Button, Piezo Siren
Node #3	`Meshtastic_af88`	`123456`	SX1262 LoRa (868MHz), NEO-6M GPS, MPU6050, SOS Button, Piezo Siren

2. QUICK SETUP INSTRUCTIONS (3 STEPS)

Step A: Add Dependencies to `pubspec.yaml`

```
dependencies:
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.34.0    # Cross-platform BLE
  permission_handler: ^11.3.1   # Location & Bluetooth permissions
```

Step B: Copy the Ready-to-Use Service Class

Copy the file `meshtastic_ble_service.dart` into your project's `lib/services/` folder. It includes:

- Auto-scanner filtering for `Meshtastic_*` devices.
- Automated GATT subscription to `FromRadio` and `ToRadio`.
- Stream `onGpsReceived`: Emits latitude, longitude, and altitude from NEO-6M GPS.
- Stream `onSosAlert`: Emits instant alerts when the physical button on D6 is pressed.
- Method `triggerAcousticSiren()`: Wirelessly fires the 3V piezo buzzer on the lost person's node.

Step C: UI Usage Example

```
final bleService = MeshtasticBleService();

// 1. Scan and connect to our node
bleService.scanForNodes().listen((results) {
  for (var r in results) {
    if (r.device.platformName.startsWith("Meshtastic_")) {
      bleService.connect(r.device);
      break;
    }
  }
});

// 2. Bind GPS coordinates to your map screen
bleService.onGpsReceived.listen((gps) {
  print("Victim Location: Lat ${gps.latitude}, Lon ${gps.longitude}");
  // Update your Google Map / Flutter Map marker!
});

// 3. Sound the physical buzzer on the node
ElevatedButton(
  onPressed: () => bleService.triggerAcousticSiren(),
  child: const Text("SOUND RESCUE SIREN"),
);
```

3. MASTER PROMPT FOR AI CODING ASSISTANTS (CURSOR / COPILOT / CLAUDE)

Tip for Teammate: If you use Cursor, GitHub Copilot, ChatGPT, or Claude on your laptop, copy the block below and paste it directly into your chat. The AI will write the exact integration code for your specific screens and state management!

📋 COPY-PASTE PROMPT FOR YOUR IDE CHAT:

```
You are a senior Flutter engineer working on our mobile Search-and-Rescue (SAR) application.
Our hardware team has built and flashed 3 custom LoRa mesh nodes based on the Seeed Studio XIAO ESP32-C6 running
Meshtastic firmware.

Hardware Details:
- Advertising Names: Filter for "Meshtastic_*" (e.g., Meshtastic_9448, Meshtastic_aee8, Meshtastic_af88)
- Default Bluetooth Pairing PIN: "123456"
- Core Service UUID: 6ba1b218-15a8-461f-9fa8-5dcae273eafd
- FromRadio (Notify/Read): 2c55e69e-4993-11ed-b878-0242ac120002
- ToRadio (Write): f75c76d2-129e-11ed-861d-0242ac120002

We have a service file `meshtastic_ble_service.dart` providing:
1. `Stream get onGpsReceived` (latitude, longitude, altitude)
2. `Stream get onSosAlert` (emergency alerts from physical button)
3. `Future triggerAcousticSiren()` (sounds the 3V piezo buzzer on the node)
4. `Stream get connectionState`

Your Task:
1. Inspect our existing Flutter project screens and state management architecture (Provider, Riverpod, or Bloc).
2. Create or update our Map/Dashboard screen to display live node markers when `onGpsReceived` emits.
3. Add a floating action button or UI card labeled "Trigger SAR Siren" that calls `triggerAcousticSiren()`.
4. Implement a high-priority dialog or banner when `onSosAlert` emits, centering the map onto the victim coordinates.
5. Provide clean, modular code with complete error handling.
```

4. BLE GATT REFERENCE TABLE

GATT Characteristic	UUID	Direction	Purpose
Meshtastic Service	`6ba1b218-15a8-461f-9fa8-5dcae273eafd`	Primary Service	Filter UUID for BLE discovery.
ToRadio	`f75c76d2-129e-11ed-861d-0242ac120002`	App → Node (Write)	Transmit siren triggers, waypoints, and configs.
FromRadio	`2c55e69e-4993-11ed-b878-0242ac120002`	Node → App (Notify)	Stream incoming GPS coordinates, SOS alerts, telemetry.
FromNum	`ed9da18c-a800-4f66-a670-aa7547e34453`	Node → App (Notify)	Sequence counter notifying app of waiting packets.